PROJECT REPORT

AegisWeave: An Evidence-Aware, ML-First Network Intrusion Detection System

Open-set attack attribution, event-time behavioural analysis and controlled simulated response, evaluated on synthetic and real network traffic

Submitted by

[Student name]

[Register / roll number]

In partial fulfilment of the requirements of

[Degree programme and department]

[Institution]

Under the guidance of

[Supervisor name and designation]

September 2026

Replace the orange placeholders before submission.

Abstract

Machine-learning network intrusion detection systems are frequently reported with near-perfect accuracy on benchmark datasets, yet they often fail silently on new networks and present unfamiliar attacks as confidently labelled known ones. This project designed, implemented and evaluated **AegisWeave**, a flow-based intrusion detection system in which machine learning is the central intelligence but every decision is separated from any response action. Network flows from Zeek logs or a JSON API pass through validated telemetry, 17 flow features and 16 causal host-behaviour features, and five complementary layers: a temperature-calibrated gradient-boosting classifier, a benign-only Isolation Forest, a temporal model, a behavioural profile and a class-support check for open-set attribution. A stacked fusion model combines them. A decision procedure distinguishes known attacks, suspicious activity, anomalous behaviour of uncertain type and explicit abstention; risk scoring, explanations, incident correlation and a policy-gated, simulation-only response engine follow.

The system was evaluated at the level an analyst actually sees, not only by binary scores. On synthetic host episodes, fusion reached F1 0.995; correcting the inherited decision logic raised the share of benign flows called normal from 35.5 % to 89.1 %, and all 337 flows of an attack family withheld from training were reported as anomalous with no invented attack type. On 1,204,888 real IoT-23 Zeek records, within-capture F1 was 0.992 but fell to 0.803 on flow vectors never seen in training; 935 of 936 flows of a withheld attack class were reported as anomalous. On captures from an unseen network and malware family, every command-and-control flow was detected (recall 1.000), but all were given the wrong attack type, the fused detector's false-positive rate rose to 10.3 %, and 30.9 % of sampled benign flows raised alerts. A live replay of those captures through the running system reproduced the offline results. The findings show that open-set decision logic prevents invented labels, but cross-network generalization remains the principal limitation and requires per-site calibration.

Keywords: network intrusion detection, machine learning, open-set recognition, anomaly detection, Zeek, IoT-23, UNSW-NB15, explainability, evaluation methodology.

Declaration and Acknowledgement of Assistance

I declare that this report describes the AegisWeave project and that all experimental results reported here were produced by the project code and data referenced in the text, and can be regenerated with the commands in Appendix A.

Parts of the design, implementation, debugging and evaluation were carried out with the assistance of AI coding assistants (OpenAI Codex/GPT and Anthropic Claude). Their outputs were executed, tested and verified against measured data before inclusion.

> *Edit this declaration to match your institution's academic-integrity policy on AI assistance, and add acknowledgements (supervisor, department, dataset authors) as required.*

Contents

Abstract i

Declaration and Acknowledgement of Assistance ii

1 Introduction 1

1.1 Motivation 1

1.2 Problem Statement 1

1.3 Objectives 1

1.4 Scope and Safety Constraints 2

1.5 Contributions 2

1.6 Report Organisation 2

2 Background and Related Work 3

2.1 Intrusion Detection and Flow Monitoring 3

2.2 Machine Learning for NIDS and Its Pitfalls 3

2.3 Open-Set Recognition 3

2.4 Models, Calibration and Fusion 3

2.5 Datasets 3

3 Requirements and System Design 5

3.1 Functional Requirements 5

3.2 Non-Functional Requirements 5

3.3 Architecture 5

3.4 Telemetry Schema and Trust Boundaries 6

3.5 Separation of Detection and Response 6

4 Implementation 7

4.1 Technology Stack 7

4.2 Telemetry Collection 7

4.3 Behaviour Engine and Event-Time Reordering 8

4.4 Feature Engineering 8

4.5 ML Intelligence Layers 9

4.6 Decision Logic and Open-Set Attribution 9

4.7 Risk Scoring 10

4.8 Explainability 10

4.9 Incidents and Storage 10

4.10 API, Authentication and Dashboard 11
4.11 Model Lifecycle and Drift Monitoring 11
5 Evaluation Methodology 12
5.1 Research Questions 12
5.2 Datasets and Preparation 12
5.3 Leakage Controls and Split Protocols 13
5.4 Metrics 13
5.5 Reproducibility 14
6 Results 15
6.1 Synthetic Episodes 15
6.2 UNSW-NB15 16
6.3 IoT-23 Real Traffic 16
6.4 Live Replay of Unseen Captures 17
6.5 Performance 18
6.6 Software Verification 18
7 Discussion 19
7.1 Answers to the Research Questions 19
7.2 What the Headline Numbers Mean 19
7.3 Where Open-Set Logic Helped and Where It Did Not 19
7.4 Why Cross-Network Transfer Failed 19
7.5 Data-Quality Lessons 20
7.6 Threats to Validity 20
7.7 Ethics and Safety 20
8 Conclusion and Future Work 21
References 22
Appendix A Reproduction Commands 23
Appendix B Test and Verification Coverage 24
Appendix C API Surface 25
Appendix D Development History 26

If page numbers are not shown, right-click the table of contents and choose Update Field.

1 Introduction

1.1 Motivation

Network intrusion detection systems (NIDS) watch network traffic for signs of attack. Signature-based systems recognise known attack patterns reliably but cannot recognise what they have not been told about. Machine-learning (ML) approaches promise to generalise from examples, and published work routinely reports detection accuracy above 99 % on public datasets. In operational use, however, ML-based NIDS are known to suffer from high false-alarm rates, poor transfer to networks that differ from the training data, and a tendency to assign confident but wrong labels to traffic unlike anything seen before [1], [2]. An analyst who is told "port scan, 100 % confidence" about a novel beaconing implant is worse off than one who is told "anomalous behaviour, attack type uncertain".

This project therefore treats ML as the core intelligence of an intrusion detection system while making three commitments: decisions must say what the evidence does and does not support, every result must be measured rather than asserted, and no model output may directly execute a network action.

1.2 Problem Statement

Build a complete, working, flow-based NIDS in which machine learning performs detection, anomaly and behavioural analysis, and attack attribution; in which uncertain and novel traffic is explicitly distinguished from known attacks; in which detection, risk and response are separated by policy and safety controls; and evaluate it honestly on synthetic and real traffic, including traffic from a network the models never saw.

1.3 Objectives

1. Ingest network flow telemetry from a validated API and from Zeek connection logs.
2. Engineer per-flow and causal host-behaviour features without using addresses, identifiers or labels as model inputs.
3. Combine supervised classification, anomaly detection, temporal and behavioural analysis through calibrated fusion.
4. Distinguish known attacks, suspicious activity, anomalous behaviour of uncertain type, and explicit uncertainty.
5. Explain every decision with measured evidence and compute a separate, policy-relevant risk score.
6. Correlate alerts into incidents and route any response through a policy engine with approval, allowlists, expiry, rollback, rate limits, a kill switch and an audit trail, in simulation only.
7. Provide an authenticated API and an analyst dashboard backed exclusively by real application data.
8. Evaluate with leakage controls and decision-level metrics on synthetic data, UNSW-NB15 and real IoT-23 Zeek traffic, including cross-network generalization and a live replay.

1.4 Scope and Safety Constraints

- **Response is simulated by design.** No firewall, host or device is ever modified; the only response adapter records simulated actions in an audited ledger, and simulation cannot be disabled through the API.
- **No live packet capture or probing.** Telemetry comes from existing flow logs, public research datasets and explicitly labelled synthetic generators.
- **Research, not production certification.** Results describe the datasets and protocols reported here and do not establish performance on any other network.

1.5 Contributions

1. **An end-to-end, tested ML-NIDS** from telemetry to dashboard, with five complementary ML layers, stacked fusion and a simulation-only response engine behind explicit safety policy.
2. **Open-set decision logic** that separates detection from attribution: attack types are shown only when a flow lies inside the training support of that class, and unverifiable attributions are never treated as known attacks.
3. **Decision-level evaluation** through the exact serving path, measuring abstention, unattributed novelty and invented attack types that binary metrics cannot show.
4. **Real-traffic engineering for Zeek:** exact handling of unset fields, bounded event-time reordering (Zeek emits connections when they end), and exclusion of unrepresentable records.
5. **A real-data evaluation on IoT-23** with three protocols (temporal blocks, withheld attack class, leave-capture-out), duplicate-vector reporting and a live replay of never-seen captures scored against ground truth.
6. **Honest reporting of failures:** inflated within-capture results, poor cross-network transfer, a dataset labelling artifact and a scaling defect found and fixed during evaluation.

1.6 Report Organisation

Chapter 2 reviews background and related work. Chapter 3 states requirements and the system design, and Chapter 4 describes the implementation. Chapter 5 sets out the evaluation methodology and Chapter 6 reports results. Chapter 7 discusses their meaning, limitations and threats to validity, and Chapter 8 concludes with future work. Appendices give reproduction commands, test coverage, the API surface and the development history.

2 Background and Related Work

2.1 Intrusion Detection and Flow Monitoring

Intrusion detection systems are commonly divided into misuse (signature) detection, which matches known attack patterns, and anomaly detection, which models normal behaviour and reports deviations. Network monitors such as Bro/Zeek [3] summarise traffic into connection records containing endpoints, ports, protocol, duration, packet and byte counters and connection state. Flow records are compact, avoid storing payloads and remain usable when traffic is encrypted, which makes them a practical basis for ML-based detection at the cost of losing payload-level evidence.

2.2 Machine Learning for NIDS and Its Pitfalls

Sommer and Paxson [1] argued that intrusion detection differs from classic ML applications: attacks are rare and diverse, errors are expensive, network traffic is highly variable, and evaluation on closed datasets rarely reflects deployment. Arp et al. [2] later catalogued recurring pitfalls in ML security research, including data snooping, spurious correlations, sampling bias, inappropriate baselines, lab-only evaluation and inappropriate metrics. This project adopts several of their recommendations directly: disjoint and time-ordered splits, reporting results on test rows whose feature vectors never occur in training, ablations, decision-level metrics, and an explicit threats-to-validity analysis.

2.3 Open-Set Recognition

A conventional classifier assigns every input to one of the classes it was trained on, even when the input belongs to none of them. Open-set recognition [4] studies classifiers that can instead reject inputs outside the known classes. In intrusion detection this matters because new malware families and attack techniques appear continually; a system that maps them onto the nearest known label produces confident but misleading alerts. AegisWeave approaches the problem with two signals: novelty relative to benign training traffic (Isolation Forest [5]) and support relative to each known attack class (a class-conditional nearest-neighbour distance calibrated on held-out flows).

2.4 Models, Calibration and Fusion

Gradient boosting [6] builds an additive ensemble of shallow trees and performs strongly on tabular features. Isolation Forest [5] isolates anomalies with random partitioning trees and needs no attack labels. Classifier probabilities are often poorly calibrated; temperature scaling [7] rescales logits with one parameter fitted on held-out data. Stacked generalization [8] trains a second-level model on the outputs of first-level models, allowing heterogeneous evidence to be combined. All models were implemented with scikit-learn [9].

2.5 Datasets

UNSW-NB15 [10] was generated at the UNSW Canberra Cyber Range, combining real normal traffic with synthetic attacks from nine families. Its designated training and testing files (175,341 and 82,332

records) do not contain host addresses or timestamps. **IoT-23** [11], published by the Stratosphere Laboratory, provides labelled Zeek connection logs from real IoT devices infected with malware and from benign honeypot devices, with addresses and timestamps intact. It therefore supports evaluation of host-behaviour and temporal features, which UNSW-NB15's CSV files cannot.

3 Requirements and System Design

3.1 Functional Requirements

Table 1: Functional requirements

ID	Requirement
F1	Accept normalized flow telemetry (single and batch) and Zeek connection logs (tab-separated and JSON).
F2	Reject malformed or hostile telemetry, including injected labels or asset priorities, before inference.
F3	Compute flow and causal host-behaviour features; never use addresses, identifiers, timestamps or labels as model inputs.
F4	Produce supervised, anomaly, temporal and behavioural evidence and a fused attack score for every flow.
F5	Decide normal, suspicious, anomalous, malicious or unknown, and attribute an attack type only when supported.
F6	Explain each decision and compute a separate risk score using trusted asset context.
F7	Persist events, correlate non-normal events into incidents and expose them through an API and dashboard.
F8	Propose, approve, expire and roll back responses under policy, in simulation only, with a complete audit trail.
F9	Verify, version and reload model artifacts; monitor distribution drift without retraining automatically.

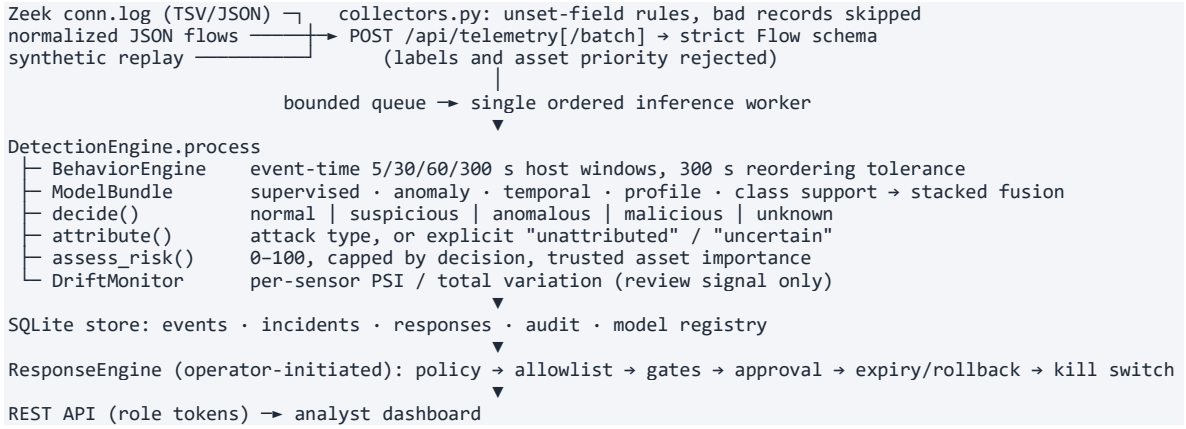
3.2 Non-Functional Requirements

- **Security:** role-based access, same-origin enforcement, bounded request sizes and rates, strict content security policy, parameterized SQL, and integrity checks before loading executable model artifacts.
- **Safety:** no path from model output to command execution; high-impact actions require a malicious, attributed decision, policy gates and human approval.
- **Honesty and explainability:** outputs state uncertainty and provenance, including whether a model was trained on synthetic or real data.
- **Reproducibility:** seeded experiments, recorded dataset hashes, model fingerprints and committed result files.
- **Performance:** per-flow inference with explanation in single-digit milliseconds on commodity hardware, with storage costs that do not grow with incident size.

3.3 Architecture

Figure 1 shows the processing pipeline. Telemetry enters through collectors or the API, is validated, queued and processed in arrival order by a single inference worker, and flows through behaviour analysis, the ML layers, decision and attribution logic, risk scoring and storage. Responses are a separate path that operators invoke explicitly and that the policy engine gates.

Figure 1: AegisWeave processing pipeline



3.4 Telemetry Schema and Trust Boundaries

Every flow is validated by a strict schema before any processing: canonical IP addresses, timezone-aware timestamps no more than five minutes in the future, bounded non-negative counters, a closed protocol vocabulary and at least one measured packet in either direction. Unknown fields are rejected, so a sender cannot supply a label, a risk score or an asset priority. Optional measurements that a sensor cannot provide (SYN ratio, connection failure, reverse counters) are recorded as missing; the model substitutes frozen benign medians and reduces evidence reliability rather than inventing values.

3.5 Separation of Detection and Response

Detection output is evidence, not a command. The response engine accepts only operator requests (block source, isolate host, notify) tied to a stored event and re-evaluates policy at proposal and approval. Containment requires a malicious decision with an attributed attack type, a risk score of at least 70, class confidence of at least 0.8, a target outside the allowlist and headroom under the action rate limit, followed by human approval. Every action carries an expiry, can be rolled back and is written to the audit trail in the same database transaction as its state change. An emergency kill switch rolls back active simulations and denies pending ones.

4 Implementation

4.1 Technology Stack

Table 2: Technology stack

Concern	Choice
Language and runtime	Python 3.12.10
Web API and server	FastAPI, Starlette, Uvicorn; pydantic 2 for validation
Machine learning	scikit-learn 1.9.1, NumPy, SciPy
Storage	SQLite in WAL mode with schema migrations and bounded retention
Dashboard	Dependency-free HTML, CSS and JavaScript served by the API
Testing	pytest; a Node-based frontend regression script; a live multi-process verification script
Deployment	Dockerfile and Compose file with a read-only, unprivileged container (not verified in this environment)

4.2 Telemetry Collection

Flows arrive as normalized JSON or from Zeek connection logs. Zeek's default output is tab-separated with typed header lines; the collector parses both that format and Zeek JSON into the same typed records, including the IoT-23 variant whose label columns are joined by spaces. Label columns in research logs are never read into telemetry. Evaluation on real logs exposed two collector defects that were corrected:

- **Unset fields.** Zeek leaves duration and payload bytes unset for single-packet connections, which are exactly the probe and scanning traffic of interest. The original collector rejected them. Table 3 lists the exact rules now applied.
- **Brittle replay.** One unrepresentable record stopped an entire replay. Invalid records are now reported and skipped, requests are batched (up to 200 flows) and rate-limit responses are honoured with backoff.

Table 3: Rules for unset Zeek measurements

Situation	Value used	Justification
Duration unset, at most one packet	0 s	Zeek leaves duration unset when first and last packet coincide; verified on all IoT-23 captures.
Payload bytes unset, zero packets	0	No packets carry no payload.
Payload bytes unset, one TCP packet	0	A lone SYN or RST probe carries no sequence-tracked payload.
Payload bytes unset, UDP/ICMP	IP bytes − packets × (IP + 8-byte header)	Exact for fixed-length headers (IPv4 20 B, IPv6 40 B).
Any other unset required value	Record rejected and reported	Values are never guessed.

4.3 Behaviour Engine and Event-Time Reordering

The behaviour engine keeps a bounded history per (sensor, simulation flag, source host) and computes causal windows of 5, 30, 60 and 300 seconds: connection counts, distinct destinations and ports, failure ratio, byte volume, destination and protocol entropy, new communication edges, rate change, activity persistence and packet-rate statistics. Inference never updates a learned baseline, so an attacker cannot gradually teach the system that attacks are normal.

Zeek writes a connection record when the connection ends but stamps it with the time it started, so records arrive out of event order. The original engine treated any record older than the newest timestamp seen as late and gave it no host history. Table 4 shows how common this is in IoT-23: up to about half of all records arrive out of order, but almost all within five minutes.

Table 4: Out-of-order arrival in IoT-23 Zeek logs

Capture	Records	Arrive out of order	Later than 300 s
Malware 1-1	1,008,748	43.2 %	0.04 %
Malware 3-1	156,103	5.8 %	0.07 %
Malware 34-1	23,145	12.3 %	1.85 %
Malware 8-1	10,403	19.9 %	0.00 %
Malware 42-1	4,426	54.9 %	0.05 %
Honeypot 5-1	1,374	51.0 %	3.20 %
Honeypot 4-1	452	46.2 %	1.77 %
Malware 44-1	237	6.8 %	0.84 %

The engine now accepts records up to 300 seconds behind the stream watermark (Zeek's TCP inactivity timeout) and inserts them in event-time order; each record sees only flows that started no later than itself, and records beyond the tolerance are isolated and cannot alter history. On capture 34-1 the share of flows with usable host history rose from 69.0 % to 95.0 %, and on honeypot 5-1 from 35.8 % to 83.8 %. For streams that arrive in order the output is unchanged, which a regression test checks by comparing features with and without the tolerance, and the synthetic confirmation results were identical before and after the change.

4.4 Feature Engineering

Features are fixed transformations with no fitted state, so training and serving compute identical values. Addresses, ports other than the destination port, identifiers, timestamps, sensor names and simulation flags are excluded.

Table 5: Model features

Group	Features
Flow (17)	log duration, packets, bytes, reverse packets, reverse bytes; log packet rate, byte rate, mean packet size; reverse packet and byte shares; protocol one-hot (TCP, UDP, ICMP, other); log destination port; connection failed; SYN ratio

Group	Features
Behaviour (16)	log connections in 5, 30, 60 and 300 s; log distinct destinations and ports in 30 s; failure ratio in 30 s; log bytes in 30 s; destination entropy; protocol entropy; new edge; edge frequency; rate change; temporal consistency; log mean and standard deviation of packet rate

4.5 ML Intelligence Layers

Table 6: ML layers and their training data

Layer	Model	Fitted on	Evidence produced
Supervised	Gradient boosting (65 depth-2 trees, balanced class weights) + temperature scaling	Training partition; temperature on calibration	Class probabilities, entropy, attack probability
Anomaly	Isolation Forest (80 trees)	Benign training flows only	Percentile of isolation score against held-out benign flows
Temporal	Histogram gradient boosting (65 iterations)	Behaviour features, training partition	Attack probability from host context
Profile	Robust (median/IQR) distance	Benign behaviour windows	Percentile deviation from benign host behaviour
Class support	5-nearest-neighbour distance per class	Training flows per class; percentiles from held-out calibration flows	How typical a flow is of the class it was assigned
Fusion	Logistic regression on 7 standardized evidence terms	Separate fusion partition	Fused attack score; threshold chosen on a tuning partition at ≤ 5 % FPR

Each layer is fitted on a different partition than the one used to calibrate or combine it, so no layer's calibration or fusion weight is learned from its own training data. Evidence reliability combines the supervised layer's calibration quality, the amount of available host history and measurement completeness; class entropy is deliberately excluded because it describes which attack type, not whether the evidence is trustworthy.

4.6 Decision Logic and Open-Set Attribution

Detection (is this an attack?) and attribution (which known attack?) are separated. Table 7 summarises the five decisions with their default thresholds.

Table 7: Decisions

Decision	Condition (defaults)	Attack type shown
malicious	Fused score ≥ 0.78, class confidence ≥ 0.70, reliability ≥ 0.50, and the flow inside the assigned class's training support	The class
suspicious	Detected or behaviourally deviating, but below the known-attack bar, or the class is unsupported or unverifiable	Tentative class or unattributed
anomalous	Anomaly percentile ≥ 0.95 and no supported known class: "Anomalous behaviour detected, attack type uncertain"	Unattributed
unknown	Reliability < 0.50, low-confidence attack evidence, or contradictory layers (explicit abstention)	Uncertain

Decision	Condition (defaults)	Attack type shown
normal	No corroborated evidence; class confidence is not required to call benign traffic normal	Normal

A flow counts as outside a class's support when its mean distance to the five nearest training flows of that class exceeds 99 % of held-out calibration flows of the same class. Evaluation on IoT-23 revealed that a rare class (DoS) had no held-out calibration flows, so no support estimate existed; the original rule treated that absence as "supported". The corrected rule treats such attributions as unverifiable: they are never malicious, and the explanation states that the label cannot be verified.

4.7 Risk Scoring

Risk (0–100) is computed after the decision from fused evidence (42 %), anomaly evidence (14 %), behavioural deviation (13 %), persistent deviation (12 %), an analyst-configured class impact prior (9 %) and trusted asset importance (10 %). Scores are capped by decision (normal 24, unknown 59, suspicious and anomalous 69, malicious 100), so uncertain or unattributed evidence can never reach the high-risk band that policy uses for containment. Severity is low below 40, medium from 40, high from 70 and critical from 85.

4.8 Explainability

- **Feature sensitivity:** each measured feature is reset to its benign training median and the model is re-run; the largest changes in attack log-odds and in isolation score are reported, explicitly labelled as sensitivity rather than causation.
- **Fusion terms:** the standardized log-odds contribution of each evidence layer to the fused score.
- **Attribution statement:** why an attack type is or is not shown, including class-support percentiles.
- **Behavioural evidence:** window counts and activity persistence, with notes when measurements were missing or history was truncated.
- **Provenance:** model identifier, version, training fingerprint, artifact hash and whether training data were synthetic.

4.9 Incidents and Storage

Non-normal events are correlated into incidents by source, sensor and simulation flag within 300-second event-time windows; an event that bridges two incidents merges them. The original implementation recomputed an incident's first and last timestamps by scanning all its events on every insert. During the live replay of a botnet capture (Section 6.4) one incident grew to 8,879 events and per-event storage time rose from 11.4 ms to 81.9 ms. Bounds are now maintained incrementally, with a randomized test confirming they always equal a full recomputation, including merges and retention deletions.

4.10 API, Authentication and Dashboard

The REST API exposes health, overview, events, incidents, models, behaviour, drift, metrics, policies, responses and audit, plus telemetry ingestion, synthetic replay, response actions, policy updates and model reload (Appendix C). Configured bearer tokens map to viewer, analyst and admin roles and are compared in constant time. Without configured tokens the server issues an ephemeral demonstration session only to loopback clients with a local host header. Mutating browser requests must be same-origin, request bodies are limited to 2 MiB, batches to 200 flows, and responses carry a strict content security policy.

The dashboard has seven views: overview, live events, ML analytics, behaviour, incidents, response and system health. Every value is loaded from the API, and all dynamic content is HTML-escaped. The event drawer shows attribution, raw classifier output, class-support percentiles, sensitivity tables, fusion terms, risk reasoning and response controls. The analytics view shows a model's own provenance warning, evaluation protocol and cross-capture results.

4.11 Model Lifecycle and Drift Monitoring

Model artifacts are saved atomically with a manifest recording the SHA-256 of the serialized bundle, the feature schema, the scikit-learn version and full metadata. Loading verifies all of these and deserializes exactly the verified bytes. Each training run records a fingerprint over code hashes, data and seed. Drift is monitored per sensor and simulation stream using population stability index and total-variation distance between a reference window and the latest window; a shift requests human review and never triggers retraining.

5 Evaluation Methodology

5.1 Research Questions

1. **RQ1:** How accurately do the individual layers and their fusion detect attacks, and how optimistic are headline scores once repeated feature vectors are excluded?
2. **RQ2:** Does the decision logic avoid alert fatigue on benign traffic while still flagging attacks?
3. **RQ3:** When an attack type is absent from training, does the system report anomalous behaviour instead of inventing a known attack type?
4. **RQ4:** How well do models trained on some networks transfer to an unseen network and malware family?
5. **RQ5:** Does the running system reproduce offline results on real traffic, at acceptable latency and throughput?

5.2 Datasets and Preparation

Synthetic host episodes. A generator produces independent host episodes (22–43 flows, the first four benign) of normal, scan, brute-force, DoS and exfiltration traffic, with a third of attack episodes deliberately subtle at flow level and periodic benign bursts. A further "unknown beacon" family is never used in training. Partitions are drawn from disjoint seeds: 80 training, 24 calibration, 24 fusion, 24 tuning and 40 test episodes (1,348 test flows).

UNSW-NB15. The designated training and testing files are preserved; a stratified 60,000-row sample of the training file is used for fitting, reduced to ten generic counters shared with the live schema. 36,516 of 82,332 test rows repeat a training feature vector, so results are reported on all rows and on the 45,816 unseen rows.

IoT-23. Eight labelled Zeek captures (1,204,888 records) were processed through the same collector and behaviour engine used in serving, in sensor arrival order. Research labels were mapped with a documented table (benign → normal, PartOfAHorizontalPortScan → scan, C&C variants → command_and_control, Attack → attack_attempt, DDoS → dos); unmapped labels are refused. 14,260 records are excluded: all belong to one DDoS flood in capture 34-1 that Zeek logged with bad checksums and zero counted packets, so they contain no traffic measurements and would teach the model an artifact. 452,276 records were re-sequenced within the reordering tolerance and 952 were isolated.

Table 8: IoT-23 captures used

Capture	Records	Represented	Labels
Honeypot 4-1	452	452	normal 452
Honeypot 5-1	1,374	1,374	normal 1,374
Malware 1-1	1,008,748	1,008,748	scan 539,465; normal 469,275; command_and_control 8
Malware 3-1	156,103	156,103	scan 145,597; attack_attempt 5,962; normal 4,536; command_and_control 8

Capture	Records	Represented	Labels
Malware 34-1	23,145	8,885	dos 14,394; command_and_control 6,706; normal 1,923; scan 122
Malware 42-1	4,426	4,426	normal 4,420; file_download 3; command_and_control 3
Malware 44-1	237	237	normal 211; command_and_control 25; dos 1
Malware 8-1	10,403	10,403	command_and_control 8,222; normal 2,181

5.3 Leakage Controls and Split Protocols

- **Synthetic:** episodes and exact feature vectors may not cross partitions (checked in code); thresholds are chosen on the tuning partition only. The seed-42 test set was inspected during development, so it is labelled exploratory; a frozen model was additionally evaluated on fresh draws (seed 381947) without refitting.
- **UNSW-NB15:** duplicate feature vectors share a validation group; test results are reported with and without rows whose vectors occur in training. An open-set test removes the four smallest families (analysis, backdoor, shellcode, worms) from training.
- **IoT-23 protocol A (temporal blocks):** each capture is cut into 20 contiguous arrival-order blocks mapped to partitions by a fixed interleaved pattern, with a 60-second event-time purge at partition boundaries. Bursty attack phases stay testable without shuffling rows.
- **IoT-23 protocol B (withheld class):** as A, with attack_attempt removed from every fitting partition.
- **IoT-23 protocol C (leave-capture-out):** captures 8-1 (Hakai botnet) and honeypot 5-1 (benign IoT devices) are never used for fitting; the remaining captures supply all fitting partitions.

IoT-23 protocols were fixed before any IoT-23 result was observed. Partition coverage gaps are reported rather than hidden: DoS has no calibration, fusion or tuning rows in any protocol, and in protocol C the fusion partition contains no command-and-control rows because capture 34-1's C&C traffic is concentrated in time.

5.4 Metrics

- **Binary detection:** precision, recall, F1, false-positive rate (FPR), ROC-AUC and PR-AUC of the fused score at the tuning-selected threshold.
- **Decision-level outcomes:** every sampled test flow is passed through the exact serving path (prediction, decision, attribution). Reported: benign flows called normal, abstaining, alerting and malicious; attacks flagged; attacks given the correct type; attacks given a wrong known type; and for each true label, which analyst-facing label it received.
- **Performance:** per-flow latency including explanation, replay throughput and storage cost per event.

5.5 Reproducibility

All experiments are seeded and write complete JSON result files, including dataset hashes, partition sizes, leakage reports and model hyperparameters. Model artifacts record training fingerprints. Appendix A lists the exact commands.

6 Results

6.1 Synthetic Episodes

Table 9: Synthetic test set: binary detection by layer

Detector	Precision	Recall	F1	FPR	ROC-AUC
Logistic regression baseline	0.978	0.765	0.859	2.22 %	0.939
Supervised layer only	0.978	0.867	0.919	2.56 %	0.952
Anomaly layer only	0.893	0.660	0.759	10.24 %	0.842
Temporal layer only	0.974	0.999	0.986	3.41 %	1.000
Fusion without temporal evidence	0.939	0.945	0.942	8.02 %	0.987
Full fusion	0.997	0.992	0.995	0.34 %	1.000

1,348 test flows. Episode-bootstrap 95 % interval for F1(full) − F1(supervised): [0.023, 0.144].

Fusion clearly outperforms every single layer, and removing temporal evidence raises FPR from 0.34 % to 8.02 %, showing that host context contributes information unavailable per flow. These are synthetic results and validate the implementation rather than real-world effectiveness.

Table 10: Synthetic test set: analyst-facing decisions

Outcome	Inherited logic	Final system	Without class support
Benign → normal	35.5 %	89.1 %	86.5 %
Benign → unknown (abstain)	64.3 %	9.6 %	8.7 %
Benign → alert	0.2 %	1.4 %	4.8 %
Attacks flagged	not measured	99.6 %	99.6 %
Attacks given the correct type	not measured	87.4 %	87.8 %
Withheld beacon family (337 flows)	26 malicious + 311 suspicious, all "scan"	337 anomalous, 0 invented types	26 malicious + 311 suspicious, all "scan"

The inherited decision logic required high class confidence even to call traffic normal, so 64.3 % of benign flows abstained, and it mapped every flow of the withheld beacon family onto "scan". Separating detection from attribution and adding the class-support check reduced abstention to 9.6 % and reported every beacon flow as anomalous. The ablation shows that on this data the class-support layer is what prevents the invented labels.

A frozen model evaluated on fresh synthetic draws without refitting gave F1 0.996, benign normal rate 89.6 %, wrong attack type 0.6 %, and 614 of 614 withheld-family flows reported as anomalous. Stress tests on the fused score alone showed that covariate shift reduced F1 only to 0.983, while on episodes containing the withheld family fused-score recall was 0.526; decision-level evaluation is what reveals that those flows are still reported as anomalous.

6.2 UNSW-NB15

Table 11: UNSW-NB15 official test set, reduced generic counters

Detector / rows	Precision	Recall	F1	FPR	ROC-AUC
Logistic baseline, all rows	0.838	0.666	0.742	15.78 %	0.793
Supervised, all rows	0.916	0.876	0.896	9.83 %	0.965
Anomaly only, all rows	0.552	0.044	0.082	4.41 %	0.691
Reduced hybrid, all rows	0.927	0.881	0.904	8.52 %	0.968
Supervised, unseen rows	0.790	0.725	0.756	12.05 %	0.914
Reduced hybrid, unseen rows	0.819	0.736	0.775	10.15 %	0.921

All rows: 82,332; unseen rows (feature vector absent from training): 45,816.

On UNSW-NB15's ten generic counters the hybrid detector reached F1 0.904 at 8.5 % FPR, falling to F1 0.775 at 10.1 % FPR on unseen vectors. Attack-family classification was weak (macro F1 0.458). In the open-set test, 6.2 % of withheld-family flows received a confident wrong type both with and without the class-support check: these families overlap known ones in the reduced feature space, so the check gave no measurable benefit here; the confidence gate withheld 91.4 % of them.

6.3 IoT-23 Real Traffic

Table 12: IoT-23 binary detection by protocol

Protocol / rows	Precision	Recall	F1	FPR	ROC-AUC
A temporal blocks, all 107,306 rows	0.995	0.988	0.992	0.70 %	0.999
A, 7,136 unseen-vector rows	0.877	0.740	0.803	6.90 %	0.913
B attack_attempt withheld, all rows	0.999	0.967	0.983	0.18 %	0.999
B, unseen-vector rows	0.928	0.303	0.457	1.56 %	0.868
C leave-capture-out, all 11,777 rows	0.957	1.000	0.978	10.27 %	0.906
C, 9,992 unseen-vector rows	0.957	1.000	0.978	20.62 %	0.810

In protocol A, 100,170 of 107,306 test rows (93 %) repeat a feature vector seen in fitting data.

Table 13: IoT-23 single layers (F1 and FPR at tuning-selected thresholds)

Protocol	Supervised F1	FPR	Anomaly F1	FPR	Temporal F1	FPR	Fusion F1
A	0.990	0.23 %	0.797	1.88 %	0.828	4.37 %	0.992
B	0.983	0.18 %	0.797	1.88 %	0.855	5.23 %	0.983
C	0.000	10.27 %	0.949	24.89 %	0.000	0.20 %	0.978

Table 14: IoT-23 analyst-facing decisions (sampled test flows)

Outcome	A	B	C
Benign flows sampled	1,773	1,773	2,000
Benign → normal	92.1 %	92.0 %	45.4 %
Benign → alert	3.2 %	3.2 %	30.9 %
Benign → malicious	0.8 %	0.4 %	0.0 %

Outcome	A	B	C
Attack flows sampled	3,559	3,559	1,000
Attacks flagged	99.8 %	99.8 %	100.0 %
Attacks given the correct type	99.2 %	73.0 %	0.0 %
Attacks given a wrong known type	0.0 %	0.0 %	100.0 %

Protocol A. Within known captures the system performs strongly (F1 0.992), but 93 % of test rows repeat fitting vectors, and on genuinely new vectors F1 falls to 0.803 with FPR 6.9 %. Destination port is the most important supervised feature (45 % of impurity importance), so the classifier partly learns capture-specific ports.

Protocol B. 935 of 936 flows of the withheld attack_attempt class were reported as anomalous, and none received an invented known type. The fused score alone detected unseen-vector attacks with recall of only 0.303; the anomaly-driven decision rule caught them. An ablation without the class-support layer produced the same 935 anomalous decisions, so on this data the benefit comes from benign-relative novelty and the decision rule rather than class support.

Protocol C. On an unseen network and malware family, detection transferred but attribution and benign precision did not. Every Hakai command-and-control flow was detected (recall 1.000), yet the supervised layer alone scored F1 0.000; detection came from the anomaly and behavioural-profile layers (fusion without the profile scored F1 0.529). All 1,000 sampled C&C flows were typed as scan. Per capture, benign FPR was 26.6 % on the unseen honeypot's devices and 0.0 % on capture 8-1's benign traffic. After the unverifiable-attribution correction (Section 4.6), no benign flow was decided malicious; before it, 260 sampled benign flows had been labelled DoS with malicious decisions (commit fc48844).

6.4 Live Replay of Unseen Captures

The protocol C model was served by the running system, and both held-out captures were replayed through the command-line collector and the authenticated API exactly as a sensor would deliver them. Ground-truth labels were joined back to stored events afterwards, for scoring only. All 11,777 events matched a labelled record.

Table 15: Live replay decisions on never-seen real traffic

True traffic	Flows	Decisions	Analyst-facing type
Hakai botnet C&C (8-1)	8,222	75.0 % suspicious, 25.0 % malicious	scan (wrong)
Benign, capture 8-1	2,181	67.1 % normal, 32.8 % uncertain	—
Benign, unseen honeypot 5-1	1,374	61.5 % anomalous, 24.7 % normal, 13.8 % uncertain	Unattributed

All benign: 50.7 % normal, 23.8 % alerting, 0.0 % malicious.

The live decision distribution for C&C traffic (75.0 % suspicious, 25.0 % malicious) agrees with the offline evaluation of the same protocol (74.8 % and 25.2 %), confirming that the serving path and the evaluation path compute the same results. No benign flow reached a malicious decision, so none became eligible for containment. Operating the response workflow on these events, a block request

for a genuinely infected host was held for human approval, while isolation of a benign smart-home device flagged as anomalous was denied because containment requires an attributed malicious decision.

6.5 Performance

Table 16: Performance measurements

Measurement	Value
Synthetic per-flow inference with explanation	p50 7.7 ms, p95 8.5 ms
IoT-23 per-flow serving path (protocol A sample)	p50 7.9 ms, p95 9.2 ms
Live replay mean inference per flow	8.6 ms
Live replay of capture 8-1 (10,403 flows), storage defect present	465 s; storage 11.4 → 81.9 ms per event
Same replay after incremental incident bounds	122 s; 11.0 → 11.5 ms per event
Replay throughput, one inference worker	≈ 85 flows per second
IoT-23 training time per protocol	20–22 s

Measured on Windows-11-10.0.26200-SP0, single-threaded model execution.

6.6 Software Verification

The automated test suite contains 136 tests across 9 files, covering schema and injection rejection, decision, attribution and risk rules, a genuinely trained model (manifest tampering, output contract, identity invariance, recomputable explanations), API authorization and perimeter controls, storage migrations and correlation, response safety, collectors and reordering, the IoT-23 corpus pipeline and an end-to-end API pipeline. A dependency-free Node script checks dashboard rendering and refresh races. A live verification script starts real server processes and runs 88 checks across 19 stages; all passed (0 failures).

7 Discussion

7.1 Answers to the Research Questions

- **RQ1.** Fusion outperforms individual layers on synthetic data, and host context adds measurable value. Headline scores are optimistic whenever test rows repeat training vectors: F1 fell from 0.992 to 0.803 on IoT-23 and from 0.904 to 0.775 on UNSW-NB15.
- **RQ2.** Separating detection from attribution removed most benign abstention (64.3 % → 9.6 % synthetic) while flagging 99.6 % of attacks. On known IoT-23 captures 92.1 % of benign flows were called normal.
- **RQ3.** Yes on synthetic data and IoT-23: withheld attack types were reported as anomalous with no invented labels. Not on UNSW-NB15's reduced counters, where withheld families resemble known ones in feature space.
- **RQ4.** Poorly. Detection of a new malware family relied on anomaly and behavioural layers and attribution failed completely; the fused false-positive rate was 10.3 %, 30.9 % of sampled benign flows raised alerts, and most flows from the unseen benign devices were reported as anomalous.
- **RQ5.** Yes. Live replay reproduced offline decisions on 11,777 real flows at single-digit millisecond inference, once a quadratic storage defect found during the replay was fixed.

7.2 What the Headline Numbers Mean

A detection F1 above 0.99 was obtained twice in this project, and in both cases it overstates what the system would achieve on new traffic: on synthetic data because the generator is hand-designed, and on IoT-23 because most test rows repeat vectors seen during fitting. Reporting only these numbers would reproduce the evaluation problems described by Arp et al. [2]. The unseen-vector results, decision-level outcomes and the leave-capture-out protocol are the more informative evidence.

7.3 Where Open-Set Logic Helped and Where It Did Not

The combination of benign-relative novelty and class support prevented invented attack types when novel traffic was genuinely different from training traffic (the synthetic beacon family; IoT-23 attack attempts). It did not help when novel classes lay close to known ones (UNSW-NB15 families in ten counters; Hakai command-and-control, which was typed as scan because it lay inside the scan class's support). Distance-based support can only flag what the feature space separates; the confidence gate and the anomaly layer carried most of the protective effect in the real-data experiments.

7.4 Why Cross-Network Transfer Failed

- **Feature reliance:** destination port dominates supervised importance, and ports are largely deployment- and malware-specific.

- **Benign diversity:** the unseen honeypot's smart-home devices used protocols and ports absent from other captures' benign traffic, so the benign-only anomaly model reported most of them as novel.
- **Small corpus:** IoT-23 contains few devices and malware samples; DoS contributes only 135 usable flows, and temporal concentration left validation partitions without some classes.

The practical implication is that a deployment should collect benign traffic from its own network to calibrate the anomaly and profile layers before the system is trusted, and should retrain and re-evaluate on locally labelled data.

7.5 Data-Quality Lessons

- Labelled public data can contain artifacts: 14,260 IoT-23 "DDoS" records had zero counted packets and would have taught the model that empty counters indicate an attack.
- Sensor semantics matter: Zeek's end-of-connection output order and unset single-packet fields silently degraded behavioural features and broke ingestion until handled explicitly.
- Evaluation at scale finds engineering defects that unit tests miss: the storage slowdown appeared only when a real botnet produced an incident of thousands of events.

7.6 Threats to Validity

- **Post-hoc correction:** the unverifiable-attribution rule was added after inspecting protocol C results. It changed decisions only, not scores, and both before and after figures are reported.
- **Exploratory synthetic test:** the seed-42 synthetic test set was inspected during development; fresh-draw confirmation mitigates but does not remove this.
- **Single seed and sampling:** IoT-23 results use one seed; decision-level figures use bounded per-label samples whose composition differs from the full population (visible in the live replay comparison).
- **Dataset scope:** synthetic data are hand-designed; UNSW-NB15 and IoT-23 are laboratory captures from 2015 and 2018–2019.
- **Deployment:** Docker packaging was not executed in this environment, and the system runs as a single process.

7.7 Ethics and Safety

The project never captured third-party traffic, never probed networks and never executed containment. Public datasets were used under their published terms, with sources cited. Response actions are simulated, policy-gated and audited, and the design prevents unattributed or uncertain evidence from reaching containment. A production deployment would additionally require legal authorization for monitoring, data-protection review, TLS, per-user identity and independent security review.

8 Conclusion and Future Work

AegisWeave demonstrates a complete, tested ML-first intrusion detection system in which detection, attribution, risk and response are deliberately separated, uncertainty is stated rather than hidden, and every reported number is reproducible from committed code and data. Its open-set decision logic prevented invented attack labels on synthetic data and on a withheld class of real IoT attacks, and a live replay reproduced offline results on real traffic.

The evaluation also shows clearly what the system cannot yet do: models trained on some laboratory networks do not transfer reliably to a new network or malware family. That conclusion, reached with leakage-aware, decision-level evaluation, is itself a central result of the project.

Future work:

1. Per-site calibration of the anomaly and behavioural profile layers on locally collected benign traffic, followed by re-measurement of cross-network false positives.
2. Reduced dependence on destination port through feature ablation and port-group abstractions.
3. Larger and more diverse corpora (additional IoT-23 captures, CIC-IDS2017 flows with timestamps, locally recorded traffic) and multiple seeds with confidence intervals.
4. Density- or representation-based open-set methods that separate classes overlapping in counter space.
5. Horizontal scaling with an external queue and database, verified container deployment and TLS.
6. Controlled live response adapters behind the existing policy engine, tested in an isolated laboratory network.

References

[1] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in Proc. IEEE Symposium on Security and Privacy, 2010, pp. 305–316.

[2] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro and K. Rieck, "Dos and don'ts of machine learning in computer security," in Proc. USENIX Security Symposium, 2022.

[3] V. Paxson, "Bro: A system for detecting network intruders in real-time," Computer Networks, vol. 31, no. 23–24, pp. 2435–2463, 1999.

[4] W. J. Scheirer, A. de Rezende Rocha, A. Sapkota and T. E. Boult, "Toward open set recognition," IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 35, no. 7, pp. 1757–1772, 2013.

[5] F. T. Liu, K. M. Ting and Z.-H. Zhou, "Isolation forest," in Proc. IEEE International Conference on Data Mining, 2008, pp. 413–422.

[6] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," Annals of Statistics, vol. 29, no. 5, pp. 1189–1232, 2001.

[7] C. Guo, G. Pleiss, Y. Sun and K. Q. Weinberger, "On calibration of modern neural networks," in Proc. International Conference on Machine Learning, 2017.

[8] D. H. Wolpert, "Stacked generalization," Neural Networks, vol. 5, no. 2, pp. 241–259, 1992.

[9] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[10] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in Proc. Military Communications and Information Systems Conference (MilCIS), 2015.

[11] S. Garcia, A. Parmisano and M. J. Erquiaga, "IoT-23: A labeled dataset with malicious and benign IoT network traffic," Stratosphere Laboratory, 2020. [Online]. Available: https://www.stratosphereips.org/datasets-iot23

Appendix A Reproduction Commands

All commands run from the repository root with the project virtual environment. Windows paths are shown; on Linux or macOS use .venv/bin/python. Indented lines continue the command above them and must be typed on the same line.

Environment and tests

```
python -m venv .venv
.venv/Scripts/python -m pip install -r requirements.lock
.venv/Scripts/python -m pytest -p no:cacheprovider
node frontend/check-ui.cjs
.venv/Scripts/python scripts/verify_pipeline.py
```

Experiments

```
.venv/Scripts/python -m experiments.run --model var/models/active
.venv/Scripts/python -m experiments.confirm
.venv/Scripts/python -m experiments.public
    --train var/datasets/UNSW_NB15_training-set.csv
    --test var/datasets/UNSW_NB15_testing-set.csv
.venv/Scripts/python -m experiments.iot23
    --model var/models/iot23 --model-lco var/models/iot23-lco
```

Demonstrations

```
.venv/Scripts/python -m aegisweave demo --count 200
.venv/Scripts/python -m aegisweave serve --port 8791
    --model var/models/iot23-lco --db var/iot23-unseen.db
.venv/Scripts/python -m aegisweave replay
    var/datasets/iot23/CTU-Honeypot-Capture-5-1.conn.log.labeled
    --format zeek-tsv --sensor honeypot-5-1 --batch-size 200
    --url http://127.0.0.1:8791
.venv/Scripts/python -m aegisweave replay
    var/datasets/iot23/CTU-IoT-Malware-Capture-8-1.conn.log.labeled
    --format zeek-tsv --sensor capture-8-1 --batch-size 200
    --url http://127.0.0.1:8791
```

The synthetic demonstration dashboard opens at http://127.0.0.1:8765 and the real-traffic replay dashboard at http://127.0.0.1:8791. IoT-23 files are downloaded from https://mcfp.felk.cvut.cz/publicDatasets/IoT-23-Dataset/IndividualScenarios/<capture>/bro/conn.log.labeled and saved as var/datasets/iot23/<capture>.conn.log.labeled; their SHA-256 hashes are recorded in experiments/results/iot23.json.

Appendix B Test and Verification Coverage

Table 17: Automated test files

File
tests/test_api_security.py
tests/test_behavior_reordering.py
tests/test_collectors.py
tests/test_corpus.py
tests/test_detection.py
tests/test_drift_isolation.py
tests/test_ml.py
tests/test_pipeline_e2e.py
tests/test_response_safety.py

Table 18: Live verification stages

Stage
1 model artifact
2 startup
3 perimeter/auth
4 telemetry validation
5 single flow layers
6 behaviour+detection
7 novel behaviour
8 batch ingestion
9 synthetic replay
10 CLI collectors
11 storage+incidents
12 risk+explanations
13 response policy
14 audit trail
15 monitoring+models
16 dashboard
17 restart persistence
18 token RBAC
19 fail closed

88 checks, 0 failures, 52.6 s.

Appendix C API Surface

Table 19: API endpoints

Method and path	Minimum role	Purpose
GET /api/health	None (perimeter applies)	Minimal health and response mode
GET /api/session	Loopback demo only	Ephemeral demonstration token
GET /api/identity	Viewer	Authenticated role
GET /api/overview, /api/metrics	Viewer	Counts, decision trend, latency, health, resources
GET /api/events, /api/events/{id}	Viewer	Filtered events and full intelligence records
GET /api/incidents, /api/incidents/{id}	Viewer	Correlated incidents and timelines
GET /api/models, /api/behavior, /api/drift	Viewer	Model registry, host windows, drift state
GET /api/policies, /api/responses	Viewer	Response policy and simulation ledger
GET /api/audit	Analyst	Audit trail
POST /api/telemetry, /api/telemetry/batch	Analyst	Ingest one or up to 200 flows
POST /api/demo	Analyst	Generate and process synthetic flows
POST /api/responses, …/approve, …/rollback	Analyst	Propose, approve or roll back simulated actions
PUT /api/policies	Admin	Update the validated response policy
POST /api/models/reload	Admin	Verify and reload the configured model artifact

Appendix D Development History

The project is under version control. Its commit history records the main development stages:

Table 20: Commits (newest first)

Commit	Summary
8e16a5d	Fix quadratic incident storage; publish IoT-23 results and live replay
fc48844	Never attribute an attack class whose support cannot be verified
26670a0	Train and evaluate the live pipeline on real IoT-23 Zeek traffic
01077bc	Harden pipeline integrity, drift isolation and model lineage
7cb6dc1	AegisWeave ML-NIDS: verified working baseline (model v1.1.0)